

CHAPTER 1

Introduction

Many real-world decision problems are so complex that they cannot be solved by hand. In such cases, *autonomous agents* that reason about these problems automatically can provide the necessary support for human decision makers. An agent is “anything that can be viewed as perceiving its environment through sensors and acting upon that environment through effectors” [Russell et al., 1995]. An artificial agent is typically a computer program—possibly embedded in specific hardware—that *takes actions* in an environment that changes as a result of these actions. Autonomous agents can act without human control or intervention, on a user’s behalf [Franklin and Graesser, 1997].

Artificial autonomous agents can assist us in many ways. For example, agents can control manufacturing machines to produce products for a company [Monostori et al., 2006, Van Moergestel, 2014], drive a car in place of a human [Guizzo, 2011], trade goods or services on markets [Ketter et al., 2013, Pardoe, 2011], and help ensure security [Tambe, 2011]. As such, autonomous agents have enormous potential to improve our productivity and quality of life.

In order to successfully complete tasks, autonomous agents require the capacity to reason about their environment and the consequences of their actions, as well as the desirability of those consequences. The field of *decision theory* uses probabilistic models of the environment, called *decision problems*, to formalize the tasks about which such agents reason. Decision problems can include the *states* the environment can be in, the possible *actions* that agents can perform in each state, and how the state is affected by these actions. Furthermore, the desirability of actions and their effects are modeled as numerical feedback signals. These feedback signals are typically referred to as *reward*, *utility*, *payoff*, or *cost* functions. Solving a decision problem consists of finding a *policy*, i.e., rules for how to behave in each state, that is optimal in some sense with respect to these feedback signals.

In most research on planning and learning in decision problems, the desirability of actions and their effects are codified in a *scalar* reward function [Busoniu et al., 2008, Oliehoek, 2010, Thiébaux et al., 2006, Wiering and Van Otterlo, 2012]. In such scenarios, agents aim to maximize the expected (cumulative) reward over time.

However, many real-world decision problems have multiple objectives. For example, for a computer network we may want to maximize performance while minimizing power consumption [Tesauro et al., 2007]. Similarly, for traffic control, we may want to maximize throughput, minimize latency, maximize fairness to drivers, and minimize noise and pollution. In response to a query, we may want a search engine to provide a balanced list of documents that maximizes

2 1. INTRODUCTION

both the relevance to the query and the readability of the documents [Van Doorn et al., 2016]. In probabilistic planning, e.g., path planning for robots, we may want to maximize the probability of reaching a goal, while minimizing the expected cost of executing the plan [Bryce, 2008, Bryce et al., 2007]. Countless other real-world scenarios are naturally characterized by multiple objectives.

In all the cases mentioned above, the problem is more naturally expressed using a vector-valued reward function. When the reward function is vector-valued, the value of a policy is also vector-valued. Typically, there is no single policy that maximizes the value for all objectives simultaneously. For example, in a computer network, we can often achieve higher performance by using more power. If we do not know the exact preferences of the user with respect to these objectives, or indeed if these preferences may change over time, it can be crucial to produce a set of policies that offer different trade-offs between the objectives, rather than a single optimal policy.

The field of *multi-objective decision making* addresses how to formalize and solve decision problems with multiple objectives. This book provides an introductory overview of this field from the perspective of artificial intelligence. In addition to describing multi-objective decision problems and algorithms for solving them, we aim to make explicit the key assumptions that underly work in this area. Such assumptions are often left implicit in the multi-objective literature, which can be a source of confusion, especially for readers new to the topic. We also aim to synthesize these assumptions and offer a coherent, holistic view of the field.

We start by explicitly formulating the motivation for developing algorithms that are specific to multi-objective decision problems.

1.1 MOTIVATION

The existence of multiple objectives in a decision problem does not automatically imply that we require specialized multi-objective methods to solve it. On the contrary, if the decision problem can be *scalarized*, i.e., the vector-valued reward function can be converted to a scalar reward function, the problem may be solvable with existing single-objective methods. Such a conversion involves two steps [Roijers et al., 2013a]. The first step is to specify a *scalarization function* that expresses the utility of the user for different trade-offs between the objectives.

Definition 1.1 A *scalarization function* f is a function that maps a multi-objective value of a policy π of a decision problem, $\mathbf{V}^\pi$, to a scalar value $V_{\mathbf{w}}^\pi$:

$$V_{\mathbf{w}}^\pi = f(\mathbf{V}^\pi, \mathbf{w}),$$

where $\mathbf{w}$ is a weight vector that parameterizes f .

The second step of the conversion is to define a single-objective version of the decision problem such that the utility of each policy π equals the scalarized value of the original multi-objective decision problem $V_{\mathbf{w}}^\pi$.

Though it is rarely stated explicitly, all research on automated multi-objective decision making rests on the premise that there are decision problems for which performing one or both of these conversion steps is impossible, infeasible, or undesirable at the moment at which planning or learning must be performed. Here, we discuss three scenarios, illustrated in Figure 1.1, where this is the case, and specialized multi-objective methods are therefore preferable. In these scenarios, we assume a planning setting. However, in Chapter 6, we briefly address the learning setting.

Figure 1.1a depicts the *unknown weights scenario*. In this scenario, $\mathbf{w}$ is unknown at the moment when planning must occur: the *planning phase*. For example, consider a company that mines different resources from different mines spread out through a mountain range. The people who work for the company live in villages at the foot of the mountains. In Figure 1.2, we depict the problem this company faces: in the morning, one van per village needs to transport workers from that village to a nearby mine, where various resources can be mined. Different mines yield different quantities of resource per worker. The market prices per unit of resource vary through a stochastic process and every price change can affect the optimal assignment of vans. Furthermore, the expected price variation increases with time. It is therefore critical to act based on the latest possible price information in order to maximize performance.

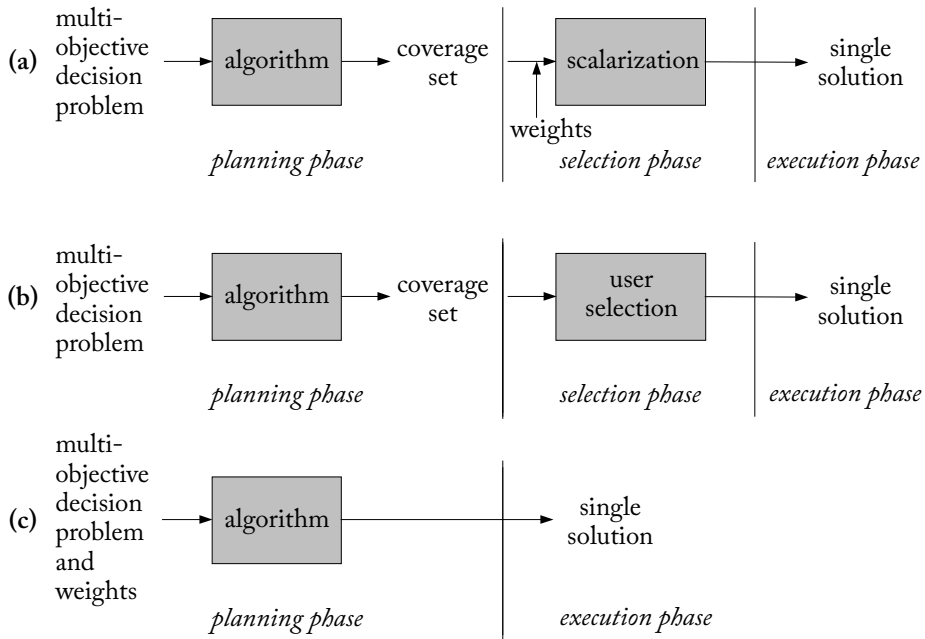


Figure 1.1: The three motivating scenarios for multi-objective decision-theoretic planning: (a) the unknown weights scenario, (b) the decision support scenario, and (c) the known weights scenario.

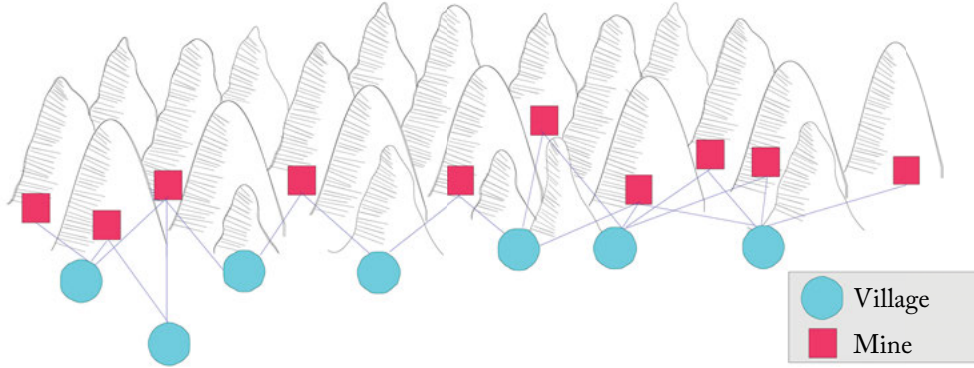


Figure 1.2: Mining company example.

Because computing the optimal van assignment takes time, computing the optimal assignment with each price change is impossible. Therefore, we need a multi-objective method that computes a set containing an optimal solution for every possible value of the prices, $\mathbf{w}$. We call such a set a *coverage set*, as it “covers” all possible preferences of the user (i.e., the possible prices) with respect to the objectives (as specified by f).¹ Although computing a coverage set is computationally more expensive than computing a single optimal policy for a given price, it needs to be done only once. Furthermore, the *planning phase* can take place in advance, when more computational resources are available.

In the *selection phase*, when the prices ($\mathbf{w}$) are revealed and we want to use as little computation time as possible, we can use the coverage set to determine the best policy by simple maximization. Finally, in the *execution phase*, the selected policy is employed.

In this book, we focus on algorithms for the planning/learning phase. For the selection phase, specialized *preference elicitation* algorithms for selecting the policy with the optimal utility from the coverage set may be necessary when the coverage set is large. For example, [Chu and Ghahramani \[2005\]](#), propose an approach to preference elicitation via Gaussian processes.

In the unknown weights scenario, *a priori* scalarization is impossible, because it would shift the burden of computation toward a point in time where it is not available. The scalarization f is known, and the weights $\mathbf{w}$ will become available in the selection phase, where a single policy is selected for execution.

By contrast, in the *decision support scenario* (Figure 1.1b), $\mathbf{w}$ and f are never made explicit. In fact, scalarization is infeasible throughout the entire decision-making process because of the difficulty of specifying $\mathbf{w}$ and/or f . For example, when a community is considering the construction of a new metro line, economists may not be able to accurately compute the economic benefit of reduced commuting times. The users may also have “fuzzy” preferences that defy meaningful

¹We provide a formal definition of the term *coverage set* in Chapter 3, Definition 3.5.

quantification. For example, if construction of the new metro line could be made more efficient by building it in such a way that it obstructs a beautiful view, then a human designer may not be able to quantify the loss of beauty. The difficulty of specifying the exact scalarization is especially apparent when the designer is not a single person but a committee or legislative body whose members have different preferences and agendas, such as the politicians and interest groups involved in constructing the metro line. In such a system, the multi-objective planning method is used to calculate a coverage set with respect to the constraints that can safely be imposed on f and $\mathbf{w}$. For example, we can safely assume that gaining value in one objective, without reducing the value in any of the others cannot reduce the utility to the user (i.e., the scalarized value).

As shown in Figure 1.1b, the decision support scenario proceeds similarly to the unknown weights scenario in the planning phase. In the selection phase, however, the user or users directly select a policy from the coverage set according to their idiosyncratic preferences, rather than explicitly computing a numerical utility by applying the scalarization function to each value vector.

In the decision support scenario, one could still argue that scalarization before planning (or learning) is possible in principle. For example, the loss of beauty can be quantified by measuring the resulting drop in housing prices in neighborhoods that previously enjoyed an unobstructed view. However, the difficulty with explicit scalarization is not only that doing so may be impractical but, more importantly, that it forces the users to express their preferences in a way that may be inconvenient and unnatural. This is because selecting $\mathbf{w}$ requires weighing hypothetical trade-offs, which can be much harder than choosing from a set of actual alternatives. This is a well understood phenomenon in the field of *decision analysis* [Clemen, 1997], where the standard workflow involves presenting alternatives *before* soliciting preferences. In the same way, algorithms for multi-objective decision problems can provide critical decision support; rather than forcing the users to specify f and $\mathbf{w}$ in advance, these algorithms just prune policies that would not be optimal for any f and $\mathbf{w}$ that fit the known constraints on the preferences of the users, and produce a coverage set. Because this coverage set contains optimal solutions for all f and $\mathbf{w}$ that fit the known constraints—rather than just all $\mathbf{w}$ for a specified f , as in the unknown weights scenario—it offers a range of alternatives from which the users can select, according to preferences whose relative importance is not easily quantified.

Finally, we present one more scenario, which we call the *known weights scenario*, that requires explicit multi-objective methods. In this scenario, we assume that $\mathbf{w}$ is known at the time of planning and thus scalarization is possible. However, it may be *undesirable* because of the difficulty of the second step in the conversion. In particular, if f is nonlinear, then the resulting single-objective problem may be much more complex than the original multi-objective problem. As a result, finding the optimal policy may be intractable while the original multi-objective problem is tractable. For example, in multi-objective Markov decision processes (MOMDPs²), which

²This abbreviation is also used for *mixed-observability MDPs* [Ong et al., 2010], which we do not consider here; we use the abbreviation MOMDPs solely for multi-objective MDPs.

6 1. INTRODUCTION

we formally define in Chapter 2, nonlinear scalarization destroys the *additivity property* on which single-objective solution methods rely (see Section 3.2.3).

Figure 1.1c depicts the known weights scenario. In contrast to the other scenarios, in the known weights scenario, the multi-objective method only produces one policy, which is then executed, i.e., there is no separate selection phase.

The scenarios we have presented here require explicit multi-objective methods because *a priori* scalarization of the multi-objective decision problems, and subsequent solving with standard single-objective methods, does not apply. In this book, we focus on the two multi-policy scenarios, i.e., the unknown weights and decision support scenarios, in which the goal of a multi-objective planning method is to produce a coverage set. From this coverage set, the policy that maximizes *user utility* is selected in the selection phase.

Of course, computing a coverage is in general more difficult than finding a single policy, and thus multi-objective methods are typically more expensive than their single-objective counterparts. It is natural then to wonder whether the complications of a multi-objective approach are merited. After all, many single-objective problems are already intractable. In this book, we take a pragmatic perspective: multi-objective methods are not a panacea and are not always the best option, even if the problem is naturally modeled with multiple objectives. If the scalarization weights are known (or can be reasonably estimated) before planning begins, and *a priori* scalarization does not yield an intractable problem, then converting a multi-objective problem to a single-objective one may be the best option. However, in any of the many cases where such a conversion is not possible or practical, then the multi-objective methods discussed in this book may prove indispensable.

1.2 UTILITY-BASED APPROACH

The goal of solving all—including multi-objective—decision problems is to maximize user utility. However, in the unknown weights and decision support scenarios, we cannot optimize user utility directly because, at the time when planning or learning takes place, the parameters $\mathbf{w}$ of the scalarization function f , which maps the multi-objective values to a scalar utility, are unknown. Therefore, we must compute a *coverage set*: a set of policies such that, for every possible scalarization, a maximizing policy is in the set (see Definition 3.5).

In this book, we argue that which policies should be included in the coverage set should be derived from what we know about f . We call this the *utility-based approach*, in contrast to the *axiomatic approach* that axiomatically assumes the coverage set is the *Pareto front*, which we define formally in Chapter 3. In short, the Pareto front is the set of all policies for which there is no other policy that has at least equal value in all objectives and has a higher value in at least one objective. Indeed, the Pareto front contains at least one optimal policy for most, if not all, scalarization functions that occur in practice. However, we argue that, while the Pareto front is *sufficient*, it is often not *necessary*. In fact, we show in Chapter 3 that the full Pareto front is required only in the special case where the scalarization function is nonlinear and policies must be deterministic.

A utility-based approach thus often results in a much smaller coverage set, which is typically less expensive to compute and easier for the user to examine.

Another advantage of the utility-based approach is that it is possible to derive how much utility is maximally lost when an exact coverage set cannot be computed [Zintgraf et al., 2015]. Such bounds are often crucial for a meaningful interpretation of the quality of approximate methods for decision-theoretic planning or learning, especially when comparing algorithms [Oliehoek et al., 2015]. Furthermore, the bounds provide insight into the quality and reliability of the selected final policy.

Multi-Objective Decision Problems

In this chapter, we introduce the concept of a multi-objective decision problem. Then we describe two concrete classes of multi-objective decision problems that we use throughout the book: multi-objective coordination graphs and multi-objective Markov decision processes. However, before introducing the concrete multi-objective decision problems, we first introduce their single-objective counterparts.

2.1 MULTIPLE OBJECTIVES

In this book, we focus on different (cooperative) multi-objective decision problems. Multi-objective decision problems contrast single-objective decision problems, which are more common in the literature. In their most general form, single-objective decision problems can be defined as a set of policies and a value function that we wish to maximize:

Definition 2.1 A cooperative *single-objective decision problem (SODP)*, consists of:

- a set of allowed (joint) *policies* Π ,
- a value function that assigns a real numbered value, $V^\pi \in \mathbb{R}$, to each joint policy $\pi \in \Pi$, corresponding to the desirability, i.e., the utility, of the policy.

Definition 2.2 In a cooperative *multi-objective decision problem (MODP)*, Π is the same as in an SODP, but

- there are $d \geq 1$ *objectives*, and
- the value function assigns a *value vector*, $\mathbf{V}^\pi \in \mathbb{R}^d$, to each joint policy $\pi \in \Pi$, corresponding to the desirability of the policy *with respect to each objective*.

We denote the value of policy π in the i -th objective as V_i^π .

Both $\mathbf{V}^\pi$ and Π may have particular forms that affect the way they should be computed, as we discuss in Chapter 3. For example, there may be multiple agents, each with its own action space. In such settings, a solution consists of a *joint policy* containing a *local policy* for each agent.

10 2. MULTI-OBJECTIVE DECISION PROBLEMS

We assume that Π is known and that it is in principle possible to compute the value of each (joint) policy. Furthermore, we assume that, if there are multiple agents, these agents are *cooperative*.

Definition 2.3 A multi-agent MODP is *cooperative* if and only if all agents get the same (team) value, $\mathbf{V}^\pi$, for executing a joint policy $\pi \in \Pi$, i.e., there are no individual rewards. A single-agent MODP is cooperative by default.

This definition of cooperative is common in the field of decision theory, e.g., in multi-agent MDPs [Boutillier, 1996, Scharpf et al., 2016] and Dec-POMDPs [Oliehoek and Amato, 2016]. However, the term “cooperative” is used differently in cooperative game theory [Chalkiadakis et al., 2011, Igarashi and Roijers, 2017], in which agents form coalitions on the basis of their individual utilities. In this book, we consider only decision problems that are cooperative according to Definition 2.3.

In an SODP, the value function provides a complete ordering on the joint policies, i.e., for each pair of policies π and π' , V^π must be greater than, equal to, or less than $V^{\pi'}$. By contrast, in an MODP, the presence of multiple objectives means that the value function $\mathbf{V}^\pi$ is a vector rather than a scalar. Such value functions supply only a partial ordering. For example, it is possible that, $V_i^\pi > V_i^{\pi'}$ but $V_j^\pi < V_j^{\pi'}$. Consequently, unlike in an SODP, we can no longer determine which values are optimal without additional information about how to prioritize the objectives, i.e., about what the *utility* of the user is for different trade-offs between the objectives.

In the *unknown weights* and *decision support scenarios* (Figure 1.1), the parameters of the scalarization function $\mathbf{w}$, or even f itself, are unknown during the planning or learning phases. Therefore, an algorithm for solving an MODP should return a set of policies that contains an optimal policy for each possible $\mathbf{w}$. Given such a solution set, the user can pick the policy that maximizes her utility in the *selection phase*. We want the solution set to contain at least one optimal policy for every possible scalarization (in order to guarantee optimality), but we also want the solution set to be as small as possible, in order to make the selection phase as efficient as possible. We discuss which solution sets are optimal, and how this can be derived from different assumptions about the scalarization function f (Definition 1.1), and the set of permitted policies Π in the MODP in Chapter 3. In the rest of this section, we introduce two different MODP problem classes.

2.2 MULTI-OBJECTIVE COORDINATION

The first class of MODPs that we treat is the *multi-objective coordination graph* (MO-CoG).¹ In a MO-CoG, multiple agents need to coordinate their actions in order to be effective. For example, in the mining problem of Figure 1.2, each agent represents a van with workers from a single village. Each of these vans can go to different mines within reasonable traveling distance, leading

¹In the literature, MO-CoGs have many different names: *multi-objective weighted constraint satisfaction problems* (MO-WCSPs) [Rollón, 2008], *multi-objective constraint optimization problems* (MO-COPs) [Marinescu, 2011], and *multi-objective collaborative graphical games* (MO-CoGG) [Roijers et al., 2013c].

to a set of different possible actions for each agent. Each mine yields a different expected amount of gold (the first objective) and silver (the second objective). Because mining can be done more efficiently when more workers are present at a mine, it is vitally important that the different agents (i.e., vans) coordinate which mines they go to.

Other examples of problems that can be modeled as a MO-CoG are: risk-sensitive combinatorial auctions, in which we want to maximize the total revenue, while minimizing the risk for the auctioneer [Marinescu, 2011], and maintenance scheduling for offices in which the energy consumption, costs, and overtime for the maintenance staff must all be minimized [Marinescu, 2011].

2.2.1 SINGLE-OBJECTIVE COORDINATION GRAPHS

Before we formally define MO-CoGs, we first define the corresponding single-objective problem, i.e., *coordination graphs* (CoGs) [Guestrin et al., 2002, Kok and Vlassis, 2004]. In the context of coordination graphs, the notion of reward is typically referred to as *payoff* in the literature. Payoff is usually denoted u (for *utility*). We adopt this terminology and notation.

Definition 2.4 A *coordination graph* (CoG) [Guestrin et al., 2002, Kok and Vlassis, 2004] is a tuple $\langle \mathcal{D}, \mathcal{A}, \mathcal{U} \rangle$, where

- $\mathcal{D} = \{1, \dots, n\}$ is the set of n agents,
- $\mathcal{A} = \mathcal{A}_1 \times \dots \times \mathcal{A}_n$ is the joint action space: the Cartesian product of the finite action spaces of all agents. A joint action is thus a tuple containing an action for each agent $\mathbf{a} = \langle a_1, \dots, a_n \rangle$, and
- $\mathcal{U} = \{u^1, \dots, u^\rho\}$ is the set of ρ scalar *local payoff functions*, each of which has limited *scope*, i.e., it depends on only a subset of the agents. The total team payoff is the sum of the local payoffs: $u(\mathbf{a}) = \sum_{e=1}^{\rho} u^e(\mathbf{a}_e)$.

In order to ensure that the coordination graph is *fully cooperative*, all agents share the payoff function $u(\mathbf{a})$. We abuse the notation e both to index a local payoff function u^e and to denote the subset of agents in its scope; $\mathbf{a}_e$ is thus a *local joint action*, i.e., a joint action of this subset of agents. The decomposition of $u(\mathbf{a})$ into local payoff functions can be represented as a *factor graph* [Bishop, 2006] (Figure 2.1); a bipartite graph containing two types of vertices: agents (variables) and local payoff functions (factors), with edges connecting local payoff functions to the agents in their scope.

The main challenge in a CoG is that the size of the joint action space $\mathcal{A}$ grows exponentially with the number of agents. It thus quickly becomes intractable to enumerate all joint actions and their associated payoffs. Key to solving CoGs is therefore to exploit *loose couplings* between agents, i.e., each agent's behavior directly affects only a subset of the other agents.

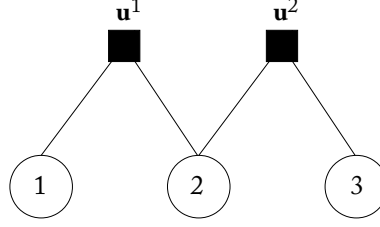


Figure 2.1: A CoG with three agents and two local payoff functions. The factor graph illustrates the loose couplings that result from the decomposition into local payoff functions. In particular, each agent’s choice of action directly depends only on those of its immediate neighbors, e.g., once agent 1 knows agent 2’s action, it can choose its own action without considering agent 3.

Figure 2.1 shows the factor graph of an example CoG in which the team payoff function decomposes into two local payoff functions, each with two agents in scope:

$$u(\mathbf{a}) = \sum_{e=1}^{\rho} u^e(\mathbf{a}_e) = u^1(a_1, a_2) + u^2(a_2, a_3).$$

The local payoff functions are defined in Table 2.1. We use this CoG as a running example throughout this book. The local payoff functions, with their limited scopes, encode the loose couplings: each agent can only directly affect another agent when they share, i.e., are both in the scope of, a local payoff function. For example, if we fix the action for agent 2 to be $\dot{a}_2$, then agents 1 and 3 can decide upon their optimal actions independently, as they do not directly affect each other.

Table 2.1: The payoff matrices for $u^1(a_1, a_2)$ (left) and $u^2(a_2, a_3)$ (right). There are two possible actions per agent, denoted by a dot ($\dot{a}_1$) and a bar ($\bar{a}_1$).

	$\dot{a}_2$	$\bar{a}_2$		$\dot{a}_3$	$\bar{a}_3$
$\dot{a}_1$	3.25	0	$\dot{a}_2$	2.5	1.5
$\bar{a}_1$	1.25	3.75	$\bar{a}_2$	0	1

2.2.2 MULTI-OBJECTIVE COORDINATION GRAPHS

We now consider the multi-objective setting:

Definition 2.5 A *multi-objective coordination graph (MO-CoG)* [Rojers et al., 2015b] is a tuple $\langle \mathcal{D}, \mathcal{A}, \mathcal{U} \rangle$ where:

- $\mathcal{D}$ and $\mathcal{A}$ are the same as in a CoG, but,
- $\mathcal{U} = \{\mathbf{u}^1, \dots, \mathbf{u}^\rho\}$ is now the set of ρ , d -dimensional local payoff functions. The total team payoff is the sum of local *vector-valued* payoffs: $\mathbf{u}(\mathbf{a}) = \sum_{e=1}^{\rho} \mathbf{u}^e(\mathbf{a}_e)$.

For example, payoffs for a MO-CoG structured as in Figure 2.1, i.e.,

$$\mathbf{u}(\mathbf{a}) = \mathbf{u}^1(a_1, a_2) + \mathbf{u}^2(a_2, a_3),$$

are provided in Table 2.2.

Table 2.2: The two-dimensional payoff matrices for $\mathbf{u}^1(a_1, a_2)$ (left) and $\mathbf{u}^2(a_2, a_3)$ (right)

	$\dot{a}_2$	$\bar{a}_2$		$\dot{a}_3$	$\bar{a}_3$
$\dot{a}_1$	(4, 1)	(0, 0)	$\dot{a}_2$	(3, 1)	(1, 3)
$\bar{a}_1$	(1, 2)	(3, 6)	$\bar{a}_2$	(0, 0)	(1, 1)

The only difference between a MO-CoG and a CoG is the dimensionality of the payoff functions. A CoG is thus a special case of a MO-CoG, i.e., a MO-CoG in which $d = 1$. Furthermore, when preferences are known, it may be possible to scalarize a MO-CoG and thus transform it into a CoG. For example, if we know the scalarization function is linear, i.e., $f(\mathbf{u}, \mathbf{w}) = \mathbf{w} \cdot \mathbf{u}$, and its parameter vector $\mathbf{w} = (0.75, 0.25)$ is, then we can scalarize the multi-objective payoff functions of Table 2.2 to the single-objective payoff functions of Table 2.1 before planning.

In a *deterministic policy* for a MO-CoG, the agents pick one joint action. In a *stochastic policy*, the agents draw a joint action from a probability distribution. Note that such a probability distribution is in general defined over *joint* actions, and there can thus still be coordination between the agents when the policy is stochastic.

When f and/or $\mathbf{w}$ are unknown in the planning or learning phase—as is the case in the *unknown weights* and *decision support* scenarios discussed in Section 1.1—we need to produce a set of policies that contains at least one optimal policy for each possible f and $\mathbf{w}$. The solution of a MO-CoG is thus a coverage set of policies. In general, this can contain both deterministic and stochastic policies. We explain why this is important for MO-CoGs (but not for CoGs) in Chapter 3.

2.3 MULTI-OBJECTIVE MARKOV DECISION PROCESSES

The second class of MODPs that we treat is the *multi-objective Markov decision process (MOMDP)*, in which a single agent needs to perform a sequence of actions over time. This sequence of actions typically takes place in an environment that is affected by these actions. Therefore, the agent has to consider not only its immediate reward, but also the reward it will be able to obtain later in the states it visits in the future.

14 2. MULTI-OBJECTIVE DECISION PROBLEMS

Consider the robot (shown as a Pacman symbol) in Figure 2.2 that needs to navigate in a socially appropriate way to reach the blue target zone in the upper right corner. We want the robot to reach the target as soon as possible, i.e., minimize the time to reach the target, but also minimize the hindrance that the robot causes to the other person by avoiding her personal space (indicated in red) along the way. By driving through the personal space of the person, it can obtain a higher value with respect to the first objective but a lower value in the second objective. Which policy is optimal thus depends on how much we value the first objective relative to the second objective.

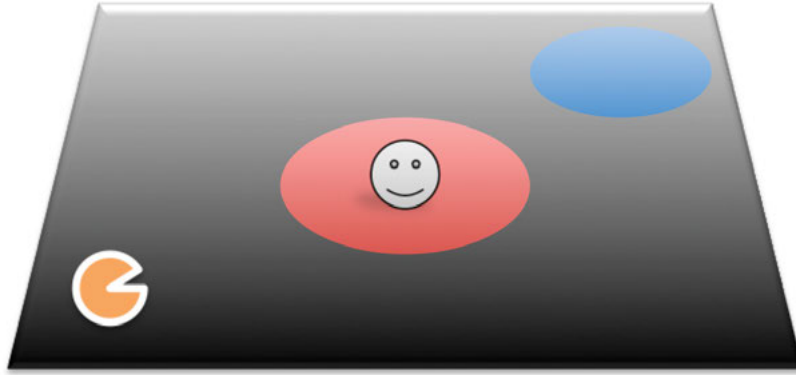


Figure 2.2: A social robot navigation problem MDP: a robot (indicated by the pacman symbol) needs to reach the target zone in the upper right corner (objective 1). However, the robots needs to avoid the personal space of the person standing in between the start position of the robot and the target zone (objective 2).

2.3.1 SINGLE-OBJECTIVE MARKOV DECISION PROCESSES

The single-objective version of an MOMDP is an MDP:

Definition 2.6 A (single-objective) *Markov decision process (MDP)* [Bellman, 1957b] is a tuple $\langle S, \mathcal{A}, T, R, \mu_0 \rangle$ where,

- S is the state space, i.e., the set of possible states the environment can be in,
- $\mathcal{A}$ is the action space, i.e., the set of actions the agent can take,
- $T : S \times \mathcal{A} \times S \rightarrow [0, 1]$ is the transition function, giving the probability of a next state given an action and a current state,
- $R : S \times \mathcal{A} \times S \rightarrow \mathbb{R}$ is the reward function, specifying the immediate expected scalar reward corresponding to a transition.

- μ_0 is the distribution over initial states s_0 .

At each timestep t , the agent observes the current state of the environment $s \in \mathcal{S}$. When the agent takes an action $a \in \mathcal{A}$ the environment transitions to a new state s' .

The state in an MDP is *Markovian*, i.e., the current state s of the environment and the current action of the agent a are a sufficient statistic for predicting the next transition probabilities $T(s'|s, a)$ and the associated expected immediate reward. The agent's history, i.e., the states and actions that led to the current state, do not provide additional information in that respect.

The agent's goal in an MDP is to find a policy π that maximizes the return R_t , which is some function of the rewards received from timestep t and onward. In the broadest sense, a policy can condition on everything that is known to the agent.

A *state-independent value function* V^π specifies the expected return when following π from the initial state:

$$V^\pi = E[R_0 \mid \pi, \mu_0]. \quad (2.1)$$

We further specify first the return, R_t , and then the policy, π .

Typically, the return is *additive* [Boutilier et al., 1999], i.e., it is a sum of the rewards attained at each timestep. When the returns are additive, V^π becomes an expectation over this sum. In a *finite-horizon setting* there is a limited number of timesteps, h , and the sum is typically undiscounted:

$$V^\pi = E\left[\sum_{t=0}^{h-1} R(s_t, a_t, s_{t+1}) \mid \pi, \mu_0\right]. \quad (2.2)$$

In a *discounted infinite-horizon setting*, the number of timesteps is not limited, but there is a discount factor, $0 \leq \gamma < 1$, that specifies the relative importance of future rewards with respect to immediate rewards:

$$V^\pi = E\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \mid \pi, \mu_0\right]. \quad (2.3)$$

Which action a_t is chosen by the agent at each timestep t depends on its policy π . If the policy is *stationary*, i.e., it conditions only on the current state, then it can be formalized as $\pi : S \times A \rightarrow [0, 1]$: it specifies, for each state and action, the probability of taking that action in that state. We can then specify the *state value function* of a policy π :

$$V^\pi(s) = E[R_t \mid \pi, s_t = s],$$

for all t when $s_t = s$. The *Bellman equation* restates this expectation recursively for stationary policies:

$$V^\pi(s) = \sum_a \pi(s, a) \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^\pi(s')]. \quad (2.4)$$

Note that the Bellman equation, which forms the heart of most standard solution algorithms such as *dynamic programming* [Bellman, 1957a] and *temporal difference methods* [Sutton and Barto,

16 2. MULTI-OBJECTIVE DECISION PROBLEMS

1998], explicitly relies on the assumption of additive returns. This is important because nonlinear scalarization functions f can interfere with this additivity property, making planning and learning methods that rely on the Bellman equation not directly applicable, as we discuss in Section 3.2.3.

State value functions induce a partial ordering over policies, i.e., π is better than or equal to π' if and only if its value is greater for all states:

$$\pi \succeq \pi' \Leftrightarrow \forall s, V^\pi(s) \geq V^{\pi'}(s).$$

A special case of a stationary policy is a *deterministic* stationary policy, in which one action is chosen with probability 1 for every state. A deterministic stationary policy can be seen as a mapping from states to actions: $\pi : S \rightarrow A$. For single-objective MDPs, there is always at least one *optimal* policy π , i.e., $\forall \pi' : \pi \succeq \pi'$, that is stationary and deterministic.

Theorem 2.7 *For any additive infinite-horizon single-objective MDP, there exists a deterministic stationary optimal policy [Boutilier et al., 1999, Howard, 1960].*

If more than one optimal policy exists, they share the same value function, known as the *optimal value function* $V^*(s) = \max_\pi V^\pi(s)$. The *Bellman optimality equation* defines the optimal value function recursively:

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]. \quad (2.5)$$

Note that, because it maximizes over actions, this equation makes use of the fact that there is an optimal deterministic stationary policy. Because an optimal policy maximizes the value for every state, such a policy is optimal regardless of the initial state distribution μ_0 . However, the state-independent value (Equation 2.1) can be different for different initial state distributions. Using μ_0 , the state value function can be translated back into the state-independent value function (Equation 2.1):

$$V^\pi = \sum_{s \in S} \mu_0(s) V^\pi(s).$$

2.3.2 MULTI-OBJECTIVE MARKOV DECISION PROCESSES

In many decision problems, such as the social robot in Figure 2.2, it is impossible, undesirable, or infeasible to define a scalar reward function, and we need a vector-valued reward function, leading to an MOMDP.

Definition 2.8 *A multi-objective Markov decision process (MOMDP) [Roijers et al., 2013a] is a tuple $\langle S, \mathcal{A}, T, R \rangle$ where,*

- $S, \mathcal{A}$, and T are the same as in an MDP, but,
- $\mathbf{R} : S \times \mathcal{A} \times S \rightarrow \mathbb{R}^d$ is now a d -dimensional reward function, specifying the expected immediate *vector-valued* reward corresponding to a transition.

MOMDPs have recently been applied to many real-world decision problems, including: water reservoir control [Castelletti et al., 2013, 2008, Giuliani et al., 2015], where policies for releasing water from a dam must be found while balancing multiple uses of the reservoir, including hydroelectric production and flood mitigation; office building environmental control [Kwak et al., 2012], in which energy consumption must be minimized while maximizing the comfort of the building’s occupants; and medical treatment planning [Lizotte et al., 2010, 2012], in which the effectiveness of the treatment must be maximized, while minimizing the severity of the side effects.

In an MOMDP, when an agent executes a policy π , its value, $\mathbf{V}^\pi$ is vector-valued, as it is an expectation of the sum over vector-valued rewards, i.e.,

$$\mathbf{V}^\pi = E\left[\sum_{t=0}^h \mathbf{R}(s_t, a_t, s_{t+1}) | \pi, \mu_0\right],$$

in the *finite-horizon setting*, and,

$$\mathbf{V}^\pi = E\left[\sum_{t=0}^{\infty} \gamma^t \mathbf{R}(s_t, a_t, s_{t+1}) | \pi, \mu_0\right].$$

in the *infinite horizon setting*.

In an MOMDP, we must find a coverage set of policies. This is significantly harder than in an MDP, not only because we need to produce a coverage set, but also because several assumptions that hold in a single-objective MDP do not hold for MOMDPs in general, requiring us to consider a larger search space of policies. In the next chapter, we provide an overview of which types of policies we need to consider given different assumptions about the MOMDP.

CHAPTER 3

Taxonomy

So far, we have motivated multi-objective decision making and described two specific multi-objective decision problems. In this chapter, we discuss what constitutes an optimal solution. Unfortunately, there is no simple answer to this question, as it depends on several critical factors.

Recall that, in Chapter 1, we proposed a *utility-based* approach for deciding what constitutes an optimal solution to a multi-objective decision problem. Rather than axiomatically assuming that the Pareto front is the appropriate solution concept, we advocate for deriving the solution concept from what we know about the problem and the scalarization function.

In this chapter, we identify the appropriate solution concept for a number of multi-objective settings. In particular, we propose a taxonomy that organizes multi-objective problems in terms of their underlying assumptions. For the sake of brevity, the taxonomy focuses on the MOMDP introduced in Chapter 2. Near the end of this chapter, however, we briefly discuss how the same principles apply to MO-CoGs.

The goal of our taxonomy is to cover most research on multi-objective decision making while remaining simple and intuitive. However, some research does not fit neatly in our taxonomy. We discuss such settings in Section 3.5.

3.1 CRITICAL FACTORS

We start by identifying three critical factors that affect what constitutes an optimal solution to a multi-objective problem. After identifying these factors, we show in Section 3.2 how they lead to different solution concepts.

3.1.1 SINGLE VS. MULTIPLE POLICIES

Following the approach of Vamplew et al. [2011], we first distinguish problems in which only one policy is sought from ones in which multiple policies are sought. Which case holds depends on which of the three motivating scenarios discussed in Chapter 1 we face.

In the unknown weights and decision support scenarios, the solution consists of multiple policies. Though these two scenarios are conceptually quite different, from an algorithmic perspective they are identical. The reason is that they are both characterized by a strict separation of the decision-making process into two phases: the planning or learning phase and the execution phase (although in on-line settings, the agent may go back and forth between the two).

In the known weights scenario, $\mathbf{w}$ is known before planning or learning begins. Therefore, returning multiple policies is unnecessary. However, as mentioned in Chapter 1, scalarization can yield a single-objective MDP that is difficult to solve.

3.1.2 LINEAR VS. MONOTONICALLY INCREASING SCALARIZATION FUNCTIONS

The second critical factor is the nature of the scalarization function. In this section, we discuss two types of scalarization function: those that are linear combinations of the rewards and those that are merely monotonically increasing functions of them.

A common assumption about the scalarization function, e.g., Barrett and Narayanan [2008], Natarajan and Tadepalli [2005], is that f is linear, i.e., it computes the weighted sum of the values for each objective.

Definition 3.1 A linear scalarization function computes the inner product of a weight vector $\mathbf{w}$ and a value vector $\mathbf{V}^\pi$:

$$V_{\mathbf{w}}^\pi = \mathbf{w} \cdot \mathbf{V}^\pi. \quad (3.1)$$

Each element of $\mathbf{w}$ specifies how much one unit of value for the corresponding objective contributes to the scalarized value. For convenience, we typically assume that f computes not only a linear combination but a convex one, i.e., the elements of the weight vector $\mathbf{w}$ are all positive real numbers and constrained to sum to 1.

Linear scalarization functions are a simple and intuitive way to scalarize. One common situation in which they are applicable is when rewards can be easily translated into monetary value. For example, consider the mining task mentioned in Chapter 1, in which different policies yield different expected quantities of various minerals. If the prices per kilo of those minerals fluctuate daily, then the task can be formulated as a multi-objective problem, with each objective corresponding to a different mineral. Each element of $\mathbf{V}^\pi$ then reflects the expected number of kilos of that mineral that are mined under π and the scalarized value $V_{\mathbf{w}}^\pi$ corresponds to the monetary value of everything that is mined. $V_{\mathbf{w}}^\pi$ can be computed only when $\mathbf{w}$, corresponding to the (normalized) current price per kilo of each mineral, becomes known.

While linear scalarization functions are intuitive and simple, they are not always adequate for expressing the user's preferences. For example, suppose there are two minerals that can be mined and only three policies are available: π_1 sends the mining equipment to a location where only the first mineral can be mined, π_2 to a location where only the second mineral can be mined, and π_3 to a location where both minerals can be mined. Suppose the owner of the equipment prefers π_3 , e.g., because it at least partially appeases clients with different interests. However, it may be the case that, because the location corresponding to π_3 has fewer minerals, there is no linear scalarization function for which π_3 is optimal. Thus, the owner's preference of π_3 implies that she, implicitly or explicitly, employs a nonlinear scalarization function.

Here, we consider the case in which f can be nonlinear, and corresponds to a common notion of the relationship between reward and utility. This class of possibly nonlinear scalarizations are the *strictly monotonically increasing* scalarization functions. These functions adhere to the constraint that, if a policy is changed in such a way that its value increases in one or more of the objectives, without decreasing in any other objectives, then the scalarized value also increases.

Definition 3.2 A scalarization function f is *strictly monotonically increasing* if:

$$(\forall i, V_i^\pi \geq V_i^{\pi'} \wedge \exists i, V_i^\pi > V_i^{\pi'}) \Rightarrow (\forall \mathbf{w}, V_{\mathbf{w}}^\pi > V_{\mathbf{w}}^{\pi'}). \quad (3.2)$$

For the sake of brevity, in the remainder of this book we refer to strictly monotonically increasing scalarization functions simply as monotonically increasing. Linear scalarization functions (with strictly positive weights) are included in this class of functions. The condition on the left-hand side of (3.2) is more commonly known as *Pareto dominance* [Pareto, 1896].

Definition 3.3 A policy π *Pareto-dominates* another policy π' when its value is at least as high in all objectives and strictly higher in at least one objective:

$$\mathbf{V}^\pi \succ_P \mathbf{V}^{\pi'} \Leftrightarrow \forall i, V_i^\pi \geq V_i^{\pi'} \wedge \exists i, V_i^\pi > V_i^{\pi'}. \quad (3.3)$$

Demanding that f is monotonically increasing is quite a minimal constraint, as it requires only that, all other things being equal, getting more reward for a certain objective is always better. In fact, it is difficult to think of any f that violates this constraint without employing a highly unnatural notion of reward.

3.1.3 DETERMINISTIC VS. STOCHASTIC POLICIES

The third critical factor is whether only deterministic policies are considered or stochastic ones are also allowed. While in most applications there is no reason to exclude stochastic policies *a priori*, there can be cases when stochastic policies are clearly undesirable or even unethical. For example, if the policy determines the clinical treatment of a patient [Lizotte et al., 2010, Shortreed et al., 2011], then flipping a coin to determine the course of action may be inappropriate. We denote the set of deterministic policies Π_D and the set of stationary policies Π_S . Both sets are subsets of all policies: $\Pi_D \subseteq \Pi \wedge \Pi_S \subseteq \Pi$. Finally, the set of policies that are both deterministic and stationary is the intersection of both these sets, denoted $\Pi_{DS} = \Pi_D \cap \Pi_S$.

In single-objective problems, these factors are typically not critical. For example, in MDPs, thanks to Theorem 2.7, we can restrict our search to deterministic stationary policies, i.e., the optimal attainable value is attainable with a deterministic stationary policy: $\max_{\pi \in \Pi} V^\pi = \max_{\pi \in \Pi_{DS}} V^\pi$.

However, the situation is more complex in multi-objective settings. As we show below, in some cases, forbidding stochastic policies can affect the nature of the appropriate solution concept.

3.2 SOLUTION CONCEPTS

The three critical factors defined above can be used to organize multi-objective settings into a taxonomy, as shown in Table 3.1. Each cell indicates the solution concept that is appropriate for the corresponding setting. In this section, we step through all the cells in the taxonomy and derive the given solution concept. Before doing so, however, we start with two definitions that, in a generic way, formalize the notion of a solution to any multi-objective problem for which multiple policies are required.

Table 3.1: The multi-objective problem taxonomy showing the critical factors and the nature of the resulting optimal solution. The columns describe whether the problem necessitates a single policy or multiple ones, and whether those policies must be deterministic (by specification) or are allowed to be stochastic. The rows describe whether the scalarization function is a linear combination of the rewards or whether this cannot be assumed and the scalarization function is merely a monotonically increasing function of them. The contents of each cell describe what an optimal solution for the given setting looks like.

	Single Policy (known weights)		Multiple Policies (unknown weights or decision support)	
	Deterministic	Stochastic	Deterministic	Stochastic
Linear Scalarization	One deterministic stationary policy (1)		Convex coverage set of deterministic stationary policies (2)	
Monotonically Increasing Scalarization	One deterministic non-stationary policy (3)	One mixture policy of two or more deterministic stationary policies (4)	Pareto coverage set of deterministic non-stationary policies (5)	Convex coverage set of deterministic stationary policies (6)

Recall that, in the unknown weights and decision support scenarios, $\mathbf{w}$ is unavailable in the planning or learning phase. Consequently, a multi-objective algorithm for such settings should in general return a set containing multiple policies. Which policies should be included will depend on the other critical factors. However, even without knowing those factors, we can already con-

clude that the set should not contain *dominated* policies, i.e., it should not contain any policies that are suboptimal for all scalarizations.

Definition 3.4 The *undominated set* (U) of an MODP is the set of all policies and associated value vectors that are optimal for some $\mathbf{w}$ of a scalarization function $f \in \mathcal{F}$:

$$U(\Pi) = \left\{ \mathbf{V}^\pi : \pi \in \Pi \wedge \exists f \in \mathcal{F} \exists \mathbf{w} \forall \pi' \in \Pi \ f(\mathbf{V}^\pi, \mathbf{w}) \geq f(\mathbf{V}^{\pi'}, \mathbf{w}) \right\},$$

where Π is the set of all allowed policies and $\mathcal{F}$ is the set of all scalarization functions that a user may have. For convenience, we assume that payoff vectors in $U(\Pi)$ contain both the value vectors and associated policies.

$U(\Pi)$ contains all policies that are optimal for some f and $\mathbf{w}$. Although this set contains no policies that are dominated, it may well contain *redundant* policies. In fact, we only need a set with at least *one* optimal policy for every f and $\mathbf{w}$. We call such a lossless subset of $U(\Pi)$ a *coverage set*, as it “covers” every f and $\mathbf{w}$ with an optimal policy.

Definition 3.5 A *coverage set* (CS), $CS(\Pi)$, is a subset of $U(\Pi)$, such that for each possible $\mathbf{w}$, there is at least one optimal solution in the CS, i.e.,

$$\forall f \in \mathcal{F} \forall \mathbf{w} \exists \pi \left(\pi \in CS(\Pi) \wedge \forall \pi' \ f(\mathbf{V}^\pi, \mathbf{w}) \geq f(\mathbf{V}^{\pi'}, \mathbf{w}) \right).$$

Table 3.2: A simple example illustrating the difference between an undominated set $U(\Pi) = \{\pi_1, \pi_2, \pi_3\}$ and a coverage set $CS(\Pi)$, which could be $\{\pi_1, \pi_2, \pi_3\}$, $\{\pi_1, \pi_2\}$, or $\{\pi_2, \pi_3\}$

V_w^π	$w = \text{true}$	$w = \text{false}$
$\pi = \pi_1$	5	0
$\pi = \pi_2$	0	5
$\pi = \pi_3$	5	2
$\pi = \pi_4$	2	2

Note that a CS is not necessarily unique. Typically, we seek the smallest possible CS. Table 3.2 shows a simple example illustrating the difference between an undominated set and a coverage set. In this example, f is parameterized by only one binary weight feature $w \in \{\text{false}, \text{true}\}$. Note that this does not imply that there are two objectives. On the contrary, the number of objectives is not specified. Instead, it simply means that there are two possible scalarization functions, each corresponding to a different value of w . Π consists of four policies and the table shows the value of each policy under each scalarization. The undominated set $U(\Pi) = \{\pi_1, \pi_2, \pi_3\}$ because each of these policies has a maximal value of 5 for some w . While $U(\Pi)$ is also a coverage set

$CS(\Pi)$, it is not a minimal one. Since there are only two possible weights, it is possible to cover these weights with only two policies, one for each weight. Thus, $\{\pi_1, \pi_2\}$ and $\{\pi_2, \pi_3\}$ are also coverage sets, and are minimal.

Once we have computed a hopefully minimal coverage set, it can be used to speed up the selection of a policy during the selection phase (see Figure 1.1). For example, in the unknown weights scenario, the coverage set is computed during the planning phase, when the weights are unknown. However, the weights are revealed before the selection phase, enabling the selection of the policy with the maximal scalarized value for the given weights:

$$\pi^* = \operatorname{argmax}_{\pi \in CS(\Pi)} V_{\mathbf{w}}^{\pi}. \quad (3.4)$$

Note that performing the argmax over Π instead of $CS(\Pi)$ would yield the original scalarized optimization problem. Hence, multi-objective algorithms can be thought of as *pruning* algorithms that prune dominated policies from Π so as to simplify the selection of a particular policy during the selection phase.

In the decision support scenario, the weights are never explicitly revealed but the same interpretation nonetheless applies: the user selects a policy from $CS(\Pi)$, a process which typically gets easier as more policies from Π are pruned away.

In principle, it is possible that no policies can be pruned, and the smallest $CS(\Pi)$ is just Π . In this case, multi-objective methods offer no leverage and can even be harmful, as they may waste computational resources confirming that no policies can be pruned. However, in practice, there are typically coverage sets that are much smaller than Π , such that efficient multi-objective methods can offer great leverage.

3.2.1 CASE #1: LINEAR SCALARIZATION AND A SINGLE POLICY

We now proceed to discuss the appropriate solution concept for each of the six cases shown in Table 3.1, starting with the top-left box labeled (1). This case assumes a linear scalarization function and a single policy (i.e., known weights); whether the policies must be deterministic or not is irrelevant for this case.

It turns out that, in this case, the presence of multiple objectives poses no difficulties. Consider the case of an MOMDP: because the inner product computed by f distributes over addition, it can be pushed inside the expectation that appears in the definition of the value function (2.3). In the infinite horizon setting this leads to:

$$V_{\mathbf{w}}^{\pi} = \mathbf{w} \cdot \mathbf{V}^{\pi} = \mathbf{w} \cdot E \left[\sum_{k=0}^{\infty} \gamma^k \mathbf{r}_{t+k+1} \right] = E \left[\sum_{k=0}^{\infty} \gamma^k (\mathbf{w} \cdot \mathbf{r}_{t+k+1}) \right]. \quad (3.5)$$

Intuitively, this is the value function of π in a new single-objective MDP that at time t generates a scalar reward $\mathbf{w} \cdot \mathbf{r}_t$. Since this single-objective MDP has additive returns, it can be solved with standard methods, yielding a single policy. Due to Theorem 2.7, a deterministic stationary policy suffices.

Note that this case is the only one in the taxonomy for which multi-objective methods are not required. However, even in this case, a multi-objective approach can still be preferable, e.g., $\mathbf{V}^\pi$ may be easier to estimate than $V_{\mathbf{w}}^\pi$ in large or continuous MOMDPs where function approximation is required (see Chapter 6).

3.2.2 CASE #2: LINEAR SCALARIZATION AND MULTIPLE POLICIES

We now turn to the top-right box of Table 3.1, labeled (2), which assumes a linear scalarization function but multiple policies (i.e., unknown weights or decision support); as before, whether the policies must be deterministic or not is irrelevant.

Since we do not know $\mathbf{w}$ during planning or learning, we want to find a coverage set. If f is linear, then $U(\Pi)$, which is automatically a coverage set, consists of the *convex hull*.¹ Substituting (3.1) in the definition of the undominated set (Definition 3.4) yields the definition of the convex hull:

Definition 3.6 The *convex hull* (CH) is the undominated set for non-decreasing linear scalarizations $f(\mathbf{V}^\pi, \mathbf{w}) = \mathbf{w} \cdot \mathbf{V}^\pi$:

$$CH(\Pi) = \left\{ \mathbf{V}^\pi : \pi \in \Pi \wedge \exists \mathbf{w} \forall \pi' \quad \mathbf{w} \cdot \mathbf{V}^\pi \geq \mathbf{w} \cdot \mathbf{V}^{\pi'} \right\},$$

where $\mathbf{w}$ adheres to the simplex constraints, i.e., $\forall i \quad w_i \geq 0$ and $\sum_i w_i = 1$.

Figure 3.1 (left) illustrates the concept of a convex hull. Each point in the plot represents the multi-objective value of a given policy for a two-objective problem. The axes represent the reward dimensions. The convex hull consists of the red and blue points, connected by lines that form a convex surface. We refer to this type of visualization as the *value-space perspective*.

Given a linear f , the scalarized value of each policy is a linear function of the weights. This is illustrated in Figure 3.1 (right), where the x -axis represents the weight for dimension 0 ($\mathbf{w}[1] = 1 - \mathbf{w}[0]$), and the y -axis the scalarized value of the policies. We refer to this type of visualization as the *weight-space perspective*. To select a policy for a given $\mathbf{w}$, we need only know the values of the convex hull policies, which form the *upper surface* of the scalarized value, as illustrated by the red and blue lines, and correspond to the four convex hull policies in Figure 3.1 (left). The upper surface forms a piecewise linear and convex function. Such functions are also well-known from the literature on partially-observable Markov decision processes (POMDPs) [Kaelbling et al., 1998], whose relationship to MOMDPs we discuss in Section 5.2.1.

Like any $U(\Pi)$, $CH(\Pi)$ can contain superfluous policies. However, we can also define the *convex coverage set* (CCS) as the specification of the coverage set when f is linear. A CCS retains at least one policy from the CH that maximizes the scalarized payoff, $\mathbf{w} \cdot \mathbf{V}^\pi$, for every $\mathbf{w}$. We can

¹The term *convex hull* is overloaded. In geometry (e.g., Jarvis [1973]), the convex hull is a superset of what we mean by the convex hull in this book.

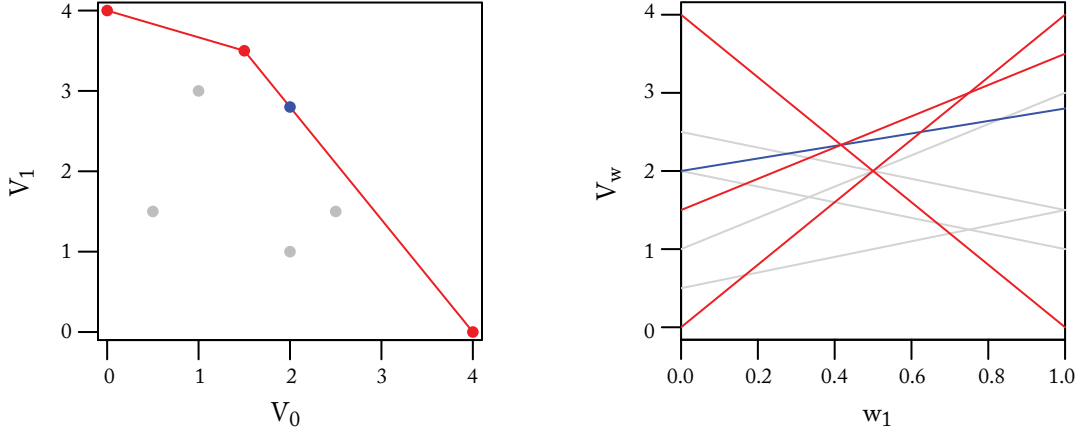


Figure 3.1: Example of the convex hull and convex coverage set. Each point at left represents the multi-objective value of a given policy. We refer to the type of visualization on the left as the *value-space perspective*. Each line at right represents the linearly scalarized value of a policy across values of $\mathbf{w}$. We refer to this type of visualization as the *weight-space perspective*. The convex hull consists of the three red points and the blue point. However, a convex coverage set can be formed using only the red points because the blue point's corresponding vector at right contributes to the upper surface but is not necessary to represent it.

obtain the definition of the CCS by substituting Equation 3.1 in the definition of the coverage set (Definition 3.5):

Definition 3.7 A *convex coverage set (CCS)*, $CCS(\Pi) \subseteq CH(\Pi)$, is a CS for linear non-decreasing scalarizations, i.e.,

$$\forall \mathbf{w} \exists \pi \left(\pi \in CCS(\Pi) \wedge \forall \pi' \mathbf{w} \cdot \mathbf{V}^\pi \geq \mathbf{w} \cdot \mathbf{V}^{\pi'} \right).$$

Figure 3.1 (left) also illustrates the difference between the CH and a CCS. In particular, a CCS can be constructed using only the red points, excluding the blue one. Figure 3.1 (right) shows why: while the blue vector (corresponding to the blue point) contributes to the upper surface, it is possible to represent that surface without the blue vector (using only the red vectors) as the blue vector is undominated at only one point and other vectors are undominated there too.

For deterministic stationary policies, the difference between $CH(\Pi)$ and $CCS(\Pi)$ may often be small. Therefore, the terms are often used interchangeably. However, in the case of non-stationary or stochastic policies, the difference is quite significant, as the CH can contain infinitely many policies, while it is possible to construct a finite CCS.

Fortunately, the following result, which is analogous to Theorem 2.7 but for MOMDPs, shows that we can restrict our attention to deterministic stationary policies.

Corollary 3.8 *For any MOMDP, a $CCS(\Pi_{DS})$ is also a $CCS(\Pi)$.*

Proof. If f is linear, we can translate the MOMDP to a single-objective MDP, for each possible $\mathbf{w}$. This is done by treating the inner product of the reward vector and $\mathbf{w}$ as the new rewards, and leaving the rest of the problem as is. Since the inner product distributes over addition, the scalarized returns remain additive (3.5). Thus, for every $\mathbf{w}$ there exists a translation to a single-objective MDP, for which an optimal deterministic and stationary policy must exist, due to Theorem 2.7. Hence, for each $\mathbf{w}$ there exists an optimal deterministic stationary policy. Therefore, there exists a $\pi \in CCS(\Pi_{DS})$ that is optimal for that $\mathbf{w}$. Consequently, there cannot exist a $\pi' \in \Pi \setminus \Pi_{DS}$ such that $\mathbf{w} \cdot \mathbf{V}^{\pi'} > \mathbf{w} \cdot \mathbf{V}^{\pi}$ and thus $CCS(\Pi_{DS})$ is also a $CCS(\Pi)$. $\square$

3.2.3 CASE #3: MONOTONICALLY INCREASING SCALARIZATION AND A SINGLE DETERMINISTIC POLICY

We now turn to cases where the scalarization is not necessarily linear but instead can only be assumed to be monotonically increasing. We start with box (3) in Table 3.1, the case for which a single deterministic policy is required.

The question then arises: Can we restrict our attention to stationary policies, as in Case #1? In that setting, we were able to push the scalarization inside the expectation computed by the value function (see (3.5)) and then use Theorem 2.7 to show that deterministic stationary policies suffice.

Unfortunately, this trick no longer works if f is not known to be linear, because nonlinear scalarization and expectation do not commute:

$$V_{\mathbf{w}}^{\pi} = f(\mathbf{V}^{\pi}, \mathbf{w}) = f\left(E\left[\sum_{k=0}^{\infty} \gamma^k \mathbf{r}_{t+k+1}\right], \mathbf{w}\right) \neq E\left[\sum_{k=0}^{\infty} \gamma^k f(\mathbf{r}_{t+k+1}, \mathbf{w})\right]. \quad (3.6)$$

Consequently, the scalarized value can no longer be written as a sum, i.e., nonlinear scalarization destroys the additivity property. We can thus no longer appeal to Theorem 2.7 to justify ignoring non-stationary policies. In fact, the following theorem shows that, for this setting, non-stationary policies may be better than the best stationary ones.

Theorem 3.9 *In infinite-horizon MOMDPs, deterministic non-stationary policies can Pareto-dominate deterministic stationary policies that are undominated by other deterministic stationary policies [White, 1982].*

To see why, consider the following MOMDP adapted from an example by White [1982]. There is only one state and three actions a_1 , a_2 , and a_3 , which yield rewards (3, 0), (0, 3), and (1, 1), respectively. If we allow only deterministic stationary policies, then there are three possible

policies $\pi_1, \pi_2, \pi_3 \in \Pi_{DS}$, each corresponding to always taking one of the actions, none of which Pareto-dominate each other. These policies have the following state-independent values:

$$\mathbf{V}^{\pi_1} = (3/(1 - \gamma), 0), \quad (3.7)$$

$$\mathbf{V}^{\pi_2} = (0, 3/(1 - \gamma)), \quad (3.8)$$

$$\mathbf{V}^{\pi_3} = (1/(1 - \gamma), 1/(1 - \gamma)). \quad (3.9)$$

However, if we now consider the set of possibly non-stationary policies Π_D , we can construct a policy $\pi_{ns} \in \Pi_D \setminus \Pi_{DS}$ that alternates between a_1 and a_2 , starting with a_1 , and whose value is:

$$\mathbf{V}^{\pi_{ns}} = (3/(1 - \gamma^2), 3\gamma/(1 - \gamma^2)). \quad (3.10)$$

Consequently, $\pi_{ns} \succ_P \pi_3$ when $\gamma \geq 0.5$ and thus we cannot restrict our attention to stationary policies.² Figure 3.2 shows the multi-objective value of each policy for different values of γ .

3.2.4 CASE #4: MONOTONICALLY INCREASING SCALARIZATION AND A SINGLE STOCHASTIC POLICY

The next case we consider is exactly the same as the one above save that stochastic policies are permitted: the scalarization function remains potentially nonlinear and a single policy is sought.

In this case, we again cannot consider only deterministic stationary policies. However, we can employ stochastic stationary policies instead of deterministic non-stationary ones. In particular, we can employ a *mixture policy* [Vamplew et al., 2009] π_m that takes a set of N deterministic policies, and selects the i -th policy from this set, π_i with probability p_i , where $\sum_{i=0}^N p_i = 1$. This leads to values that are a linear combination of the values of the constituent policies. In our previous example, we can replace π_{ns} by a policy π_m that chooses π_1 with probability p_1 and π_2 otherwise, resulting in the following values:

$$\mathbf{V}^{\pi_m} = p_1 \mathbf{V}^{\pi_1} + (1 - p_1) \mathbf{V}^{\pi_2} = \left(\frac{3p_1}{1 - \gamma}, \frac{3(1 - p_1)}{1 - \gamma} \right).$$

If $\frac{1}{3} \leq p_1 \leq \frac{2}{3}$, then $\pi_m \succ_P \pi_3$. Hence, a suitable solution for this setting is a mixture policy of two or more deterministic stationary policies.³

3.2.5 CASE #5: MONOTONICALLY INCREASING SCALARIZATION AND MULTIPLE DETERMINISTIC POLICIES

We now consider cases where the scalarization function is only assumed to be monotonically increasing but where we seek multiple policies, starting with the deterministic case.

²White [1982] shows this in an infinite-horizon discounted setting, but the arguments hold also for the finite-horizon and average-reward settings.

³A mixture policy is both stochastic and non-stationary, as it conditions on something other than the state when picking its action: the outcome of the random event that determines which of the constituent policies to execute. In theory, it is also possible to require stationary policies without requiring deterministic policies [Clemptner, 2016].

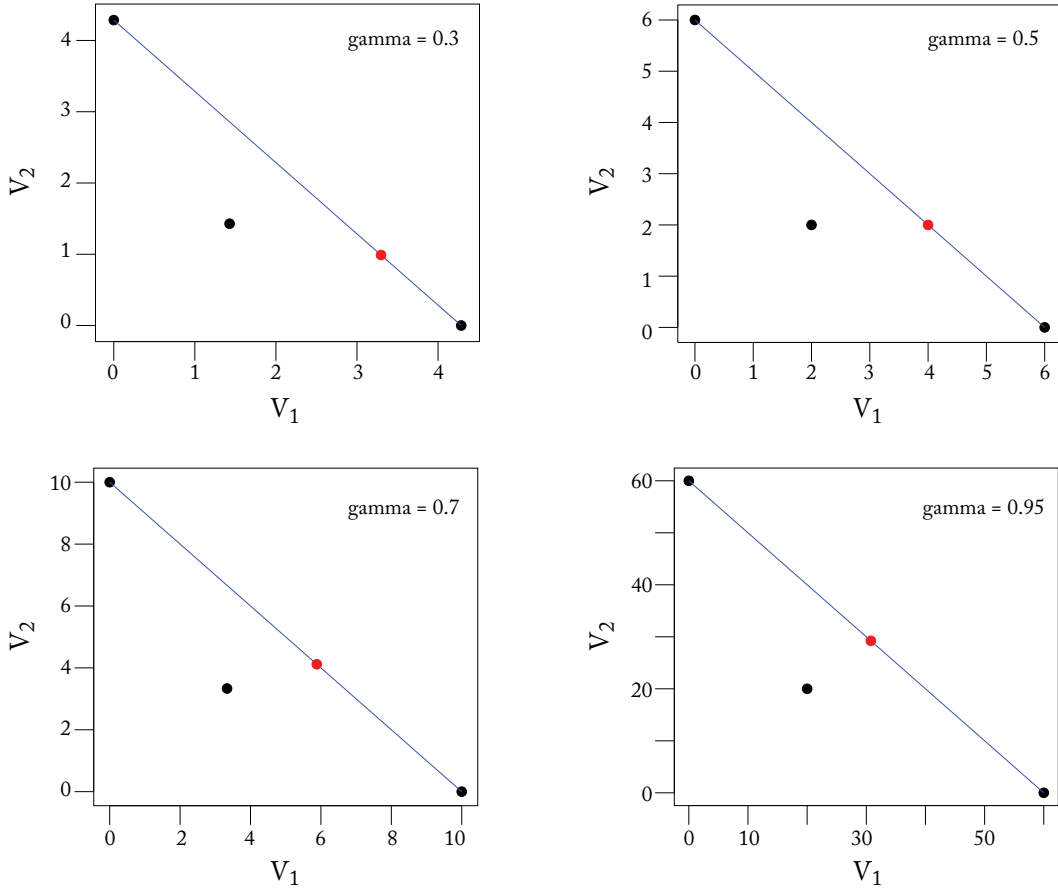


Figure 3.2: The multi-objective value of the deterministic stationary policies (black dots) in White's example and the deterministic non-stationary policy (red dot) for different values of γ .

Just as we specialized the undominated set to the case of linear scalarization functions, yielding the convex hull, we can also specialize the undominated set to the case of monotonically increasing scalarization functions, yielding the *Pareto front*.

Definition 3.10 The *Pareto front* (PF) is the set of all policies that are not Pareto-dominated:

$$PF(\Pi) = \left\{ \pi : \pi \in \Pi \wedge \neg \exists \pi' \in \Pi \mathbf{V}^{\pi'} \succ_P \mathbf{V}^{\pi} \right\}.$$

Figure 3.3 uses the same scenario depicted in Figure 3.1 to illustrate an example of a Pareto front. As before, the red and blue points together comprise the CH. Now, the addition of the black

point yields a Pareto front. The black point belongs in the PF because no other point Pareto-dominates it. This can be seen by noting that the region to the upper right of the black point (marked with the dashed lines) contains no points. The black point does not belong in the CH because it does not contribute to the undominated upper surface shown at right.

A point can be Pareto optimal while not contributing to this upper surface because Pareto dominance is pairwise: a single policy must be found that is superior for all weights. In the example, for every $\mathbf{w}$, there exists a vector with higher scalarized value than the black vector, which is why it is not in the CH, but there is no single vector that does so for all $\mathbf{w}$, which is why it is in the PF.

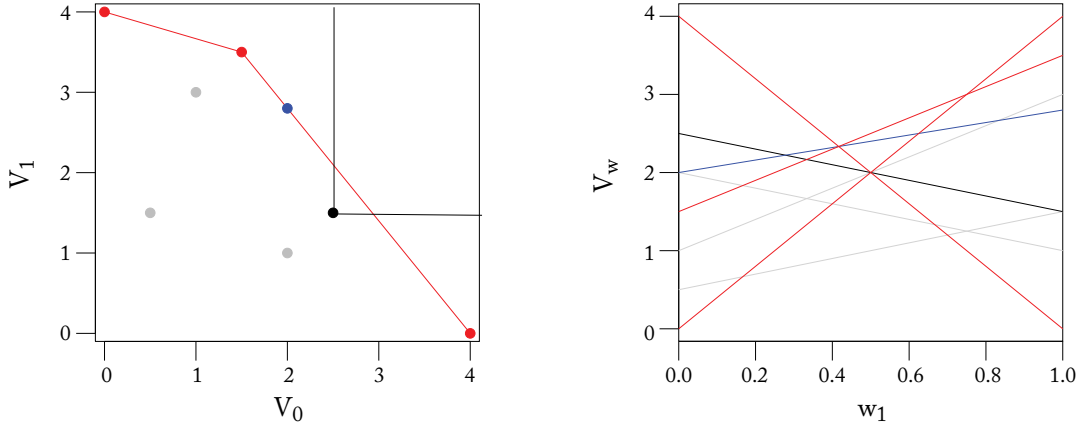


Figure 3.3: Example of the Pareto front. Each point at left represents the multi-objective value of a given policy and each line at right represents the linearly scalarized value of a policy across values of $\mathbf{w}$. A minimal CCS can be made from the three red points. The addition of the blue point forms the CH. The addition of the black point forms the Pareto front. Since no point is to the upper right of the black point (in the region marked with the dashed lines), it is not Pareto-dominated. However, since the figure at right shows that it does not contribute to the undominated upper surface, it is not in the convex hull.

Note that $PF(\Pi)$ is not the set of undominated policies $U(\Pi)$ for all specific strictly monotonically increasing f . We have already seen that for the special case of linear f , $U(\Pi) = CH(\Pi)$, which is a subset of $PF(\Pi)$. However, for any strictly monotonically increasing f , we know that a policy that is not in the $PF(\Pi)$ is dominated with respect to f , i.e., $\pi \notin PF(\Pi) \Rightarrow \pi \notin U(\Pi)$. This is because, for strictly monotonically increasing f and $\pi \notin PF(\Pi)$, there cannot exist a $\mathbf{w}$ for which π is optimal, since by definition there exists a π' such that $\mathbf{V}^{\pi'} \succ_P \mathbf{V}^{\pi}$ and, since f is strictly monotonically increasing, this implies that $V_{\mathbf{w}}^{\pi'} > V_{\mathbf{w}}^{\pi}$.

However, if we know only that f is strictly monotonically increasing, we cannot settle for a subset of $PF(\Pi)$ either, because there exist strictly monotonically increasing f for which

$U(\Pi^m) = PF(\Pi^m)$. Perny and Weng [2010] show that $U(\Pi) = PF(\Pi)$ for the *Tchebycheff function*, which is strictly monotonically increasing. Therefore, we cannot discard policies from the $PF(\Pi)$ and retain an undominated set $U(\Pi)$ for all strictly monotonically increasing f .

Like any undominated set, $PF(\Pi)$ may contain redundant policies. Therefore, we define the *Pareto coverage set* (PCS), a special case of the coverage set for monotonically increasing scalarization functions:

Definition 3.11 A *Pareto coverage set* (PCS), $PCS(\Pi) \subseteq PF(\Pi)$, is a lossless subset of $PF(\Pi)$, i.e., it only needs to contain each unique value-vector in the PF once:

$$\mathbf{V}^\pi = \mathbf{V}^{\pi'} \implies \pi \in PCS(\Pi) \vee \pi' \in PCS(\Pi) \vee \pi \notin PF(\Pi).$$

Again, for deterministic stationary policies, the difference between a $PCS(\Pi)$ and $PF(\Pi)$ may be minor. Note that $PF(\Pi)$ is automatically a $PCS(\Pi)$. Most papers in the literature therefore take $PF(\Pi)$ as the solution concept.

3.2.6 CASE #6: MONOTONICALLY INCREASING SCALARIZATION AND MULTIPLE STOCHASTIC POLICIES

We now consider the sixth and final case: monotonically increasing scalarization functions with multiple stochastic policies. A natural solution concept for this case is a PCS of stochastic stationary policies. As in the single policy settings, a monotonically increasing scalarization function that might not be linear implies that we cannot consider only stationary deterministic policies. However, if stochastic policies are allowed, we do not need to consider non-stationary policies. The result is a PCS of stochastic stationary policies.

However, because this set is in general not finite, it poses practical problems for any algorithm that hopes to compute it. Fortunately, it is not necessary to explicitly represent an entire $PCS(\Pi)$. Instead, it is sufficient to compute a $CCS(\Pi_{DS})$, i.e., a convex coverage set of deterministic stationary policies. The necessary stochastic policies to create a $PCS(\Pi)$ can then be easily constructed by making mixture policies, as in Section 3.2.4, from those policies in the $CCS(\Pi_{DS})$.

Corollary 3.12 In an infinite horizon discounted MOMDP, an infinite set of mixture policies P_M can be constructed from policies that are on a $CCS(\Pi_{DS})$, such that this set P_M is a $PCS(\Pi)$ [Vamplew et al., 2009].

Proof. We can construct a policy with any value vector on the convex surface, e.g., the red lines in Figure 3.3 (left), by mixing policies on a $CCS(\Pi_{DS})$, e.g., the red dots.⁴ Therefore, we can always

⁴We should always mix policies that are “adjacent”; the line between any pair of the policies we mix should be on the convex surface. For example, mixing the policy represented by the leftmost red dot in Figure 3.3 (left) and the policy represented by the rightmost red dot does not lead to optimal policies, as the line connecting these two points is under the convex surface.

construct a mixture policy that dominates a policy with a value under this surface, such as the black dot. We can show by contradiction that there cannot be any policy above the convex surface. If there was, it would be optimal for some $\mathbf{w}$ if f was linear. Consequently, by Corollary 3.8, there would be a deterministic stationary policy with at least equal value. But since the convex surface spans the values on the $CCS(\Pi_{DS})$, this leads to a contradiction. Thus, no policy can Pareto-dominate a mixture policy on the convex surface. $\square$

Thanks to Corollary 3.12, it is sufficient to compute a $CCS(\Pi_{DS})$ to solve an MOMDP. A surprising consequence of this fact, which to our knowledge is not made explicit in the literature, is that Pareto optimality, though the most common solution concept associated with multi-objective problems, is actually only necessary in one specific problem setting:

Observation 3.13 The multiple policy setting when f is monotonically increasing and only deterministic policies are considered (box (5) of Table 3.1) requires computing a Pareto coverage set. When either f is linear or stochastic policies are allowed, a $CCS(\Pi_{DS})$ suffices.

Wakuta [1999] proves the sufficiency of a $CCS(\Pi_{DS})$ for monotonically increasing scalarizations with multiple stochastic policies in infinite horizon MOMDPs, but using a different approach than Corollary 3.12. Instead of mixture policies, he uses stationary randomizations over deterministic stationary policies. Wakuta and Togawa [1998] provide a similar proof for the average reward case.

Note that, while it is common to consider non-stationary or stochastic policies when f is nonlinear, such policies typically condition only on the current state, or the current state and time, not the agent’s reward history. However, in this setting, policies that condition on that reward history can dominate those that do not. For example, suppose there are two objectives which can take only positive values and f simply selects the smaller of the two, i.e., $f(\mathbf{V}^\pi, \mathbf{w}) = \min_i V_i^\pi$. Suppose also that, in a given state, two actions are available, which yields rewards of (4, 4) and (0, 5) respectively. Finally, suppose that the agent can arrive at that state with one of two reward histories, whose discounted sums are either (5, 0) or (3, 3). A policy that conditions on these discounted reward histories can outperform policies that do not, i.e., the optimal policy selects the action yielding (4, 4) when the reward history sums to (3, 3) and the action yielding (0, 5) when the reward history sums to (5, 0). So, while for single objective MDPs the Markov property and additive returns are sufficient to restrict our attention to policies that ignore history, in the multi-objective case, the scalarized returns are no longer additive and therefore the optimal policy can depend on the history.

3.3 IMPLICATIONS FOR MO-COGS

The preceding discussion of the taxonomy focused on MOMDPs. In this section, we briefly discuss the implications for MO-CoGs. Overall, the situation is quite similar to that of MOMDPs. The main difference is that there is no notion of a non-stationary policy, since a MO-CoG is not a sequential problem.

In particular, given known weights and a linear scalarization function (box 1), we again do not require special methods, as the MO-CoG can be converted to a CoG by scalarizing each local payoff function, i.e., we can move $\mathbf{w}$ inside the expectation, just as in (3.5) but where the inner summation goes over the local payoff functions instead of timesteps:

$$u_{\mathbf{w}}^{\pi} = \mathbf{w} \cdot \mathbf{u}^{\pi} = \mathbf{w} \cdot E \left[\sum_{k=1}^{\rho} \mathbf{u}^k(\mathbf{a}) \right] = E \left[\sum_{k=1}^{\rho} \mathbf{w} \cdot \mathbf{u}^k(\mathbf{a}) \right], \quad (3.11)$$

where π specifies a distribution over the joint action $\mathbf{a}$. As with any CoG, a deterministic policy suffices. If the weights are not known (box 2), a $CCS(\Pi_D)$ suffices.

Given known weights but a monotonically increasing scalarization function, then a stochastic policy may be preferable, for the same reasons as in an MOMDP. If stochastic policies are allowed (box 4), we can construct one using mixture policies. If deterministic policies are required (box 3), then we cannot mimic a mixture policy with a non-stationary one, and thus the best deterministic policy may have less value than the best stochastic policy.

If the weights are not known, then a similar logic applies. If stochastic policies are permitted (box 6), then a $CCS(\Pi_D)$ suffices as it can be used to construct the needed mixture policies. Otherwise (box 5), we need a $PCS(\Pi_D)$. For some weights, the best policy in that $PCS(\Pi_D)$ may be inferior to some mixture policies that could be formed from the $CCS(\Pi_D)$, were that permitted.

3.4 APPROXIMATE SOLUTION CONCEPTS

So far, we have only discussed *exact* solution concepts, e.g., a coverage set contains an *optimal* policy for each $\mathbf{w}$. However, in many settings, computing such exact solutions may be infeasible. Therefore, in this section, we discuss approximate variants of the PCS and CCS. We start with a helpful definition:

Definition 3.14 For a given solution set S and some family of scalarization functions $\mathcal{F}$, the maximum utility loss $MUL(S, \mathcal{F})$ is the maximum scalarized value that is lost due to approximation:

$$\forall f \in \mathcal{F} \quad \forall \mathbf{w} \quad \forall \mathbf{V}^{\pi} \in CS(\Pi) \quad \exists \mathbf{V}^{\pi'} \in S \quad f(\mathbf{V}^{\pi}, \mathbf{w}) \leq f(\mathbf{V}^{\pi'}, \mathbf{w}) + MUL(S, \mathcal{F}),$$

where $CS(\Pi)$ is the coverage set appropriate for $\mathcal{F}$.

Several approximate variants of the PCS have been proposed. One of the most popular is the ε -PCS [Zitzler et al., 2003].⁵ There are multiple definitions of the ε -PCS; here, we provide the definition of the *additive* ε -PCS. There is also a *multiplicative* ε -PCS [Zintgraf et al., 2015].

Definition 3.15 A given solution set S is an additive ε -PCS if

$$\forall \mathbf{V}^{\pi} \in PCS(\Pi) \exists \mathbf{V}^{\pi'} \in S : \forall i = 1, \dots, d : V_i^{\pi} \leq V_i^{\pi'} + \varepsilon,$$

where d is the number of objectives.

Note that an ε -PCS may not contain any undominated solutions— $S \cap PF(\Pi)$ may be an empty set—but at least the maximal difference between a value vector in the PCS and the closest value vector in the ε -PCS is at most ε in all dimensions.

However, a fundamental limitation of an ε -PCS for arbitrary monotonically increasing scalarizations is that *any* increase in any objective may lead to an infinite increase in user utility. Therefore, it is impossible to compute the MUL of an ε -PCS without more information about f and $\mathbf{w}$. For example, if we know that f is Lipschitz-continuous with a Lipschitz-constant L , the MUL is bounded by $\varepsilon\sqrt{d}L$ [Zintgraf et al., 2015].

For linear scalarization functions, we have much more information, i.e., we know the exact shape of f , and that $\mathbf{w}$ adheres to the simplex constraints. In this case, we can formulate an ε -CCS for which the MUL is ε .

Definition 3.16 A given solution set S is an ε -CCS if

$$\forall \mathbf{w} \max_{\mathbf{V}^{\pi} \in CCS(\Pi)} \mathbf{w} \cdot V_i^{\pi} - \max_{\mathbf{V}^{\pi'} \in S} \mathbf{w} \cdot V_i^{\pi'} \leq \varepsilon,$$

where $\mathbf{w}$ is a linear weight vector adhering to the simplex constraints.⁶

Note that an ε -PCS is automatically an ε -CCS, though it need not be a minimal one.

3.5 BEYOND THE TAXONOMY

While the taxonomy presented in this chapter aims to cover as much research on multi-objective decision making as possible, it is nonetheless not completely comprehensive. In this section, we briefly mention a few formalisms that fall outside the scope of our taxonomy.

In particular, our taxonomy, and also the entire utility-based perspective, assumes the existence of a scalarization function f . Furthermore, we consider only monotonically increasing scalarization functions. We argue that this is a minimal assumption because it simply states that increasing the value with respect to one objective without diminishing the others cannot decrease the scalarized value. This corresponds well with what an objective is commonly understood to be.

⁵The ε -PCS is called ε -approximate Pareto front by Zitzler et al. [2003]. We use ε -PCS for consistency with our own terminology.

⁶We can always make $\mathbf{w}$ adhere to the simplex constraints by dividing by a constant c . If $\mathbf{w}$ is not on the simplex, the MUL reported here should be multiplied by this c .

However, it is possible to have a preference ordering over value vectors to which no f corresponds. For example, when preferences are based on a lexicographical ordering, i.e., even an infinitesimal improvement in an objective with a higher priority is better than a large improvement in an objective with a lower priority, and the rewards are vectors in $\mathfrak{R}^n$, there is no corresponding $f : \mathfrak{R}^n \rightarrow \mathfrak{R}$ [Sen, 1995]. Therefore, lexicographical orderings fall outside of our mathematical formalism.

Furthermore, our taxonomy considers only one special case of the monotonically increasing scalarization function, namely linear scalarization functions. However, other special cases have also been considered. For example, certain fairness constraints on f lead to the *Lorenz front* [Perny et al., 2013], a subset of the Pareto front.

